Contents lists available at ScienceDirect

Computers & Graphics

journal homepage: www.elsevier.com/locate/cag

Layer-wise Stage-aware Adapter Scaling for Multi-view Diffusion Texture Generation

ARTICLE INFO

Keywords: multi-view diffusion, 3D texturing, adapter scaling, timestep control, shape-texture trade-off

ABSTRACT

Multi-view diffusion pipelines for 3D asset texturing inject geometric conditions through adapter modules, and the strength of these residuals is usually a single global scale, although residuals act at different UNet depths and denoising stages. We study layer-wise stage-aware adapter scaling, a training-free inference control that assigns each UNet depth group its own stage-dependent residual scale, $\mathbf{h}'_{l,t} = \mathbf{h}_{l,t} + s_l(p_t)A_l(\mathbf{h}_{l,t}, \mathbf{G})$, with per-group implementation caps and no change to any network weight. On a shared-input 24-object, three-seed development ablation, depth-group redistribution improves foreground fidelity over a global fixed-low baseline, and a complete binary enumeration of temporal patterns shows that late-stage high scale helps while early-stage high scale hurts; the strongest pattern (layer-LLH) wins five of seven reported metrics, and the originally motivated layer-LHL is a representative rather than an optimum. A pre-registered strict-276-object confirmation transfers the development-selected pattern under the same runner, and a global stage-placement follow-up together with a second-backbone audit bound where stage effects replicate. We report the method as a reusable layer-wise inference control with metric-dependent trade-offs, a 12-object 3D baking case study extended to the layer-wise family with same-draw controls, and explicit negative results, rather than as a universal schedule selector.

1. Introduction

Recent advances in single-image 3D generation have substantially improved the recovery of geometry, multi-view consistency, and sparse-view reconstruction [1, 2, 3, 4, 5, 6, 7, 8]. As geometric generation models become increasingly mature, generating high-quality textures on an existing 3D shape has become a key factor affecting the realism and usability of 3D content. For an object whose geometry has already been determined, the final visual quality depends not only on whether the shape is complete, but also on whether the texture is faithful to the reference image, aligned with the 3D surface, and rich in local details.

Existing multi-view texture generation methods usually generate RGB images from multiple target views using 2D image diffusion models, and then map the generated images back to the 3D surface through projection or texture baking [9, 10, 11, 12]. This route benefits from the generative capability of image diffusion models, but it still faces challenges in reference appearance alignment, geometry-texture consistency, and local texture detail preservation. Geometry-controlled multi-view diffusion methods represented by MVPainter introduce geometric conditions such as normals and depth maps to improve the alignment between generated textures and the underlying 3D surface, demonstrating the importance of explicit geometric control for 3D texture generation [13].

However, introducing geometric conditions is not only a question of whether a control signal should be used, but also a question of how strongly it should be applied. In adapter-augmented diffusion pipelines, geometric conditions are usually injected into the model through an adapter or control branch, and their influence is regulated by a scaling factor. In practice, this factor is often set empirically: a small scale may lead to insufficient geometric constraints, whereas a larger scale is usually expected to improve structural consistency.

In multi-view texture generation, however, stronger adapter scaling is not necessarily better. Texture generation requires not only contours and structures to be consistent with geometry, but also surface color variations, material details, and high-

frequency patterns to be preserved. Overly strong geometric control may force the model to follow normal, depth, or edge conditions too aggressively, thereby suppressing the intrinsic texture synthesis capability of the base diffusion model. In this case, some structural metrics may still improve, while the visual result suffers from texture flattening, reduced color variation, and detail loss.

Therefore, adapter scaling in geometry-controlled multi-view diffusion deserves further analysis. Compared with prior work that mainly studies how to build texture generation pipelines, how to introduce geometric conditions, and how to improve overall generation quality, adapter strength is a fine-grained inference-time control problem. It directly affects the balance between shape consistency and texture detail, and is an easily overlooked factor in multi-view texture generation quality.

This problem is related to, but not solved by, existing guidance scheduling strategies. CFG scheduling changes the extrapolation between conditional and unconditional predictions, while a geometry adapter injects residuals into intermediate representations. Generic ControlNet or adapter scale scheduling similarly treats condition strength mainly as a condition-adherence knob. In texture generation, however, the key question is not only how strongly to follow the geometry, but also when geometric residuals should dominate and when they should recede so that the base model can synthesize color, material, and high-frequency texture. Simple monotonic schedules such as linear or cosine decay do not explicitly address this non-monotonic requirement. Even the recent finding that classifier-free guidance is most useful only in a limited noise interval [14] does not answer this question: it identifies where conditioning helps for CFG, but not which stages of a geometry-conditioned texture pipeline require strong geometric residuals, where the late-stage risk is texture suppression rather than diversity loss.

The goal of this paper is not to propose a new full texture generation system, but to analyze and improve inference-time adapter scaling within a fixed geometry-controlled multi-view diffusion pipeline.

The main contributions of this paper are as follows:

(1) We identify and quantify the shape-texture trade-off caused by adapter scaling in multi-view diffusion texture generation, showing why a single scale magnitude is an incomplete inference control, and we separate structure, reference-fidelity, texture-variation, and 3D-output evidence dimensions.

(2) We formulate training-free layer-wise stage-aware adapter residual scaling with an explicit scale function $s_l(p)$ over UNet depth groups and denoising stages, per-group implementation caps, and no added network forward; the method requires no retraining.

(3) We separate layer redistribution from temporal placement with shared-input paired ablations: a complete binary enumeration of all eight temporal patterns shows robust directions (late-high helps, early-high hurts), with layer-LLH, layer-LHH, and constant layer-LLL as explicit counterexamples to any layer-LHL optimum, and we confirm the development-selected pattern on a strict 276-object holdout under a pre-registered protocol.

(4) We map the evidence boundary honestly: a 12-object GLB baking and unseen-view case study extended to the layer-wise family with same-draw global controls, a global stage-placement follow-up, and a second-backbone audit that replicates the global stage effect but not yet the layer-wise method.

2. Related Work

2.1. 3D Texture Generation and Multi-view Diffusion

Single-image 3D generation is often decomposed into geometry generation and texture generation. In recent years, single-image-to-3D, multi-view diffusion, and sparse-view reconstruction methods have made notable progress in structural stability, shape completeness, and cross-category generalization [1, 2, 3, 4, 5, 6, 7, 8]. Nevertheless, high-quality texture generation remains a key bottleneck for the realism and usability of 3D content, as reflected by recent surveys of neural 3D mesh texturing [15]. Existing texture generation methods usually rely on 2D diffusion models to synthesize RGB images from multiple views, and then project or bake these images onto the 3D surface [9, 10, 11, 12]. While this strategy inherits the strong generative capability of image diffusion models, it still suffers from reference appearance misalignment, geometry-texture inconsistency, and insufficient local texture detail. Beyond these diffusion pipelines, the graphics community continues to advance surface appearance synthesis more broadly, from exemplar-based texture synthesis [16] to diffusion-based procedural material generation [17].

To address these issues, methods such as Paint3D, MVPaint, Hunyuan3D 2.0, and MVPainter formulate 3D texture generation as a geometry-controlled multi-view diffusion task, generating multi-view images under geometric control and baking them into UV textures [18, 19, 20, 13]. MV-Adapter further shows that adapter-style multi-view control can enhance cross-view consistency without fully retraining the base model [21], and recent texturing systems strengthen consistency during texture synthesis itself, for example through synchronized multi-view diffusion [22] or consistent-and-seamless text-guided multi-view generation [23]. By injecting geometric information such as normals and depth maps into the diffusion model as conditional signals, the generated results better follow the 3D surface structure and remain consistent across views. Unlike prior methods that focus mainly on multi-view image generation or general 3D reconstruction, these works emphasize precise alignment between texture and geometry as well as local detail recovery, significantly improving 3D texture generation quality. However, they mainly answer how to introduce geometric conditions and how to construct a texture generation pipeline, while the influence of geometric control strength at inference time on shape and texture quality remains underexplored.

2.2. Adapter-augmented Diffusion Models and Geometric Control

Adapter modules have become an important mechanism for introducing additional conditions into diffusion models. T2I-Adapter and ControlNet inject spatial conditions such as edges,

depth maps, and normals into a pretrained diffusion model through lightweight parallel branches with a controllable scaling factor [24, 25]. IP-Adapter introduces image prompts through decoupled attention [26]. LoRA and its variants implement lightweight model adaptation through low-rank residual updates [27, 28, 29]. Although these methods differ in architecture, they generally include some form of scaling factor that controls the influence of the external condition or adapter residual on the base model.

In practice, the adapter scale is often treated as an empirical hyperparameter. A larger scale usually improves condition adherence, for example by better following normals, depth maps, or reference images. In texture generation tasks, however, overly strong conditional constraints may weaken the base diffusion model's ability to synthesize color, material, and high-frequency details. Adapter scaling is therefore not a simple matter of increasing the strength as much as possible; it involves a balance between geometric constraints and texture fidelity. This paper focuses on exactly this inference-time control problem: given a fixed adapter structure and fixed geometric conditions, how should the adapter strength be set or scheduled to preserve both shape consistency and texture detail?

2.3. Diffusion Guidance Scheduling and Timestep Control

The denoising process of diffusion models is highly stage-dependent. Early steps mainly determine global layout and coarse structure, middle steps refine shape and semantic structure, and late steps are more responsible for color, texture, and high-frequency details [30, 31]. Based on this observation, many guidance scheduling strategies have been explored in both research and practice. For example, classifier-free guidance scales may be adjusted linearly, with cosine schedules, or dynamically to mitigate oversaturation, detail loss, or reduced diversity caused by overly strong guidance [32, 33]. In addition, the development of latent diffusion, high-resolution diffusion, and Transformer-based diffusion models further shows the close relationship among generation quality, noise schedules, conditional injection, and network architecture [34, 35, 36, 37, 38, 39, 40, 41].

These methods all exploit the staged nature of denoising to regulate conditional strength. Notably, Kynkäänniemi et al. show that classifier-free guidance is harmful at high noise levels, largely unnecessary at low noise levels, and beneficial mainly in a middle interval, motivating limited-interval guidance [14]; mid-stage concentration of conditioning strength therefore has a close precedent in CFG scheduling. However, different control signals act through different mechanisms. CFG scheduling adjusts the weight between conditional and unconditional predictions, mainly affecting the balance among semantic control, diversity, and visual fidelity. By contrast, the scale of a geometric adapter acts directly on intermediate features or residual branches, controlling how strongly geometric conditions intervene in the generation process. Adapter-scale scheduling of the form studied here differs from limited-interval guidance in both mechanism and target: it scales adapter feature residuals rather than score extrapolation, raises selected stages or depth groups above a conservative baseline instead of switching guidance on and off, and addresses the shape–texture trade-off of geometry-conditioned texture generation, where late-stage over-control suppresses the synthesis of color and high-frequency texture. ControlNet-style scale control is closer in spirit, but it is still usually interpreted as a condition-adherence strength; directly applying a larger, smaller, or smoothly decayed control scale does not specify which denoising stage should receive strong geometric residuals for texture generation.

This distinction matters because the desired schedule for geometry-controlled texture generation is not necessarily monotonic. A monotonic decay or warm-up can reduce over-control in one part of the trajectory, but it cannot simultaneously avoid unstable early residuals, strengthen the middle stage where geometry refinement is most useful, and weaken late residuals that compete with color and high-frequency texture formation. Thus, timestep-conditioned scheduling of adapter strength cannot be treated as a direct transfer of CFG scheduling or generic ControlNet scale scheduling; it requires a dedicated analysis of how geometric residual injection interacts with texture synthesis.

3. Method

3.1. Task Definition and Base Pipeline

This paper builds on the MVPainter-style geometry-controlled multi-view texture generation framework [13] and studies how the inference-time strength of geometric condition injection affects the generated results. Figure 1 illustrates the overall pipeline: the adapter injects geometric residuals into UNet features at three depth groups, the proposed scaling assigns their strength per depth group and denoising stage, and the final multi-view outputs are projected and baked into a textured 3D mesh. Given a single reference image and the corresponding 3D geometry, the base pipeline first generates RGB texture images from six target views and then obtains a textured mesh through projection or texture baking. We do not modify the base UNet, VAE, or pretrained adapter weights. Instead, the adapter scaling factor is treated as the key inference-time control variable.

During multi-view diffusion, the 3D geometry is rendered into normal maps, depth maps, and foreground masks, denoted as:

$$\mathbf{G} = \{G_{\text{normal}}, G_{\text{depth}}, G_{\text{mask}}\}. \tag{1}$$

The geometric adapter maps these conditions to a residual correction term and injects it into the intermediate hidden states of the UNet. For the t-th denoising step, the adapter injection can be written as:

$$\mathbf{h}'_t = \mathbf{h}_t + s(p_t) \cdot A_\phi(\mathbf{h}_t, \mathbf{G}), \tag{2}$$

where $\mathbf{h}_t$ and $\mathbf{h}'_t$ denote the hidden states before and after injection, p_t denotes the sampling progress, and $s(p_t)$ is the adapter scaling strength at the current denoising stage. This formulation highlights a key difference from CFG-style guidance: changing $s(p_t)$ does not reweight two predicted scores, but changes the magnitude of a geometry-conditioned residual added to the

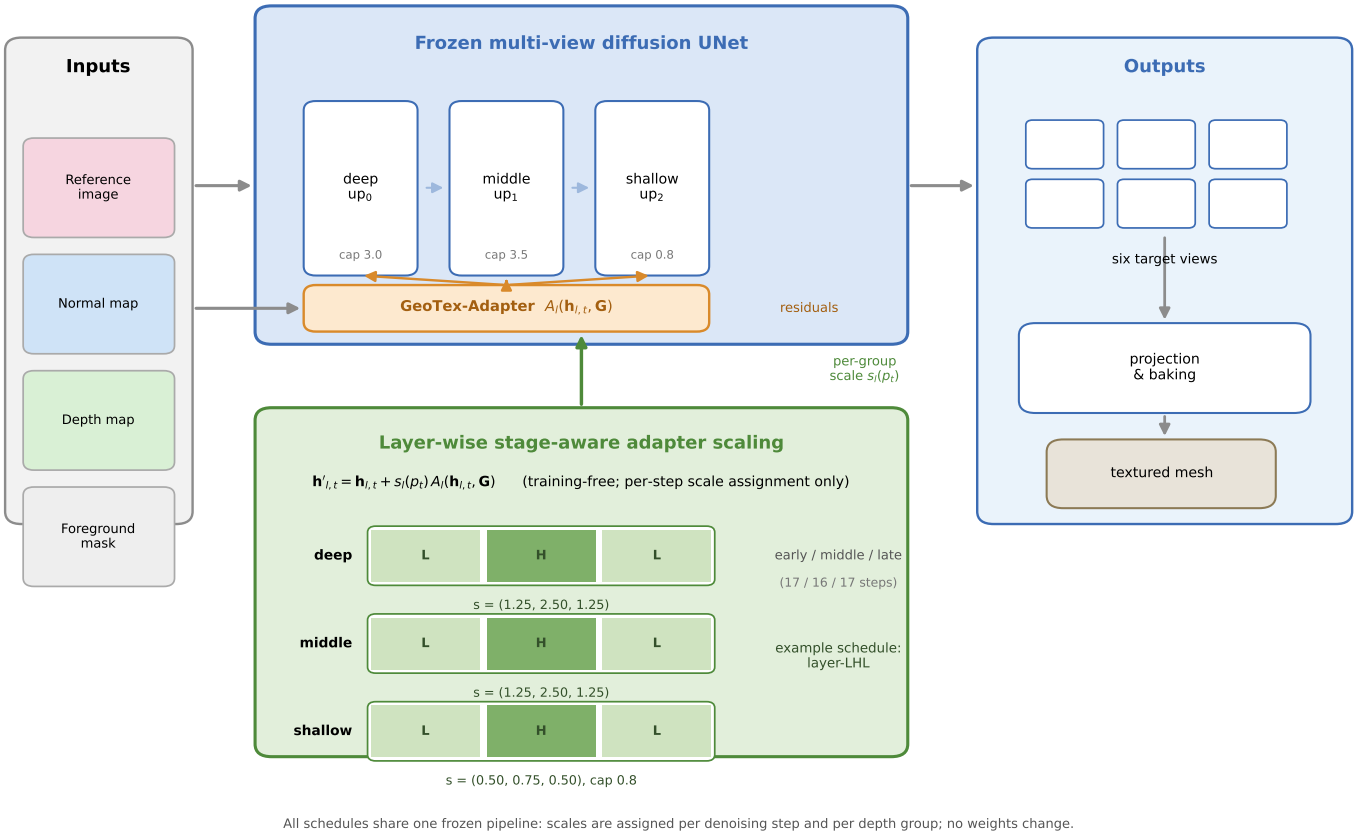


Fig. 1. Method overview. The frozen multi-view diffusion UNet receives geometry-conditioned residuals from the GeoTex-Adapter at three UNet depth groups (deep, middle, shallow). Layer-wise stage-aware adapter scaling assigns each depth group its own stage-dependent scale $s_l(p_t)$; the example shows the layer-LHL schedule and the per-group caps. Scales are assigned per denoising step; no network weight is modified. The final multi-view outputs are projected and baked into a textured 3D mesh.

feature stream. A larger s usually enhances geometric condition adherence. However, if strong residuals are continuously applied in the late denoising stage, they may suppress the synthesis of color, material, and high-frequency details by the base diffusion model. This paper therefore studies how to schedule s during denoising to balance shape consistency and texture detail preservation.

3.2. Shape-Texture Diagnostic Metrics

The influence of adapter scaling on generated results cannot be evaluated by a single structural metric. We divide evaluation metrics into three groups. The first group consists of shape and structure metrics, including FG-SSIM, Edge-SSIM, and Crop-SSIM, which measure consistency with ground-truth renderings in the foreground, edge regions, and local foreground crops based on the structural similarity index [42]. The second group contains signal fidelity and perceptual similarity metrics, including PSNR and the learned perceptual image patch similarity (LPIPS) [43], which reflect pixel-level error and differences in deep feature space. The third group consists of texture diagnostic metrics, including Laplacian variance, RGB standard deviation, and gradient magnitude, which are used to detect weakened color variation, surface smoothing, and high-frequency detail loss.

Because Reviewer-facing interpretation depends on what each metric can support, we keep four evaluation dimensions explicitly separate throughout the paper: (i) *structure* (FG-SSIM, Edge-SSIM, Crop-SSIM against ground-truth renderings); (ii) *reference fidelity* (PSNR, masked LPIPS, CIEDE2000 where available, and GT-relative errors of the variation diagnostics); (iii) *texture variation* (Laplacian variance, RGB standard deviation, gradient magnitude, high-frequency energy—descriptive only, never read as fidelity on their own); and (iv) *actual 3D output quality* (baked-mesh unseen-view metrics). In particular, a higher gradient magnitude or high-frequency energy can equally reflect unwanted noise, repeated patterns, or color shifts, so variation statements are always paired with GT-relative errors or visual counterexamples.

Taking Laplacian variance as an example, the foreground texture response is defined as:

$$M_{\text{tex}} = \text{Var}(\text{Lap}(I_{\text{out}} \odot G_{\text{mask}})), \tag{3}$$

where Lap denotes the Laplacian filter, G_{mask} denotes the foreground mask, and Var denotes spatial variance. A larger M_{tex} usually corresponds to richer local high-frequency variation. If structural metrics improve while Laplacian variance, RGB standard deviation, or gradient magnitude decreases, the improvement may mainly come from contour smoothing or boundary convergence rather than true texture quality improvement.

3.3. Stage-aware Adapter Scaling: Shared-layer TCAS and the Layer-wise Form

The diffusion denoising process is stage-dependent: early steps mainly establish the global layout and coarse contours; middle steps refine geometric structure and cross-view alignment; late steps are more responsible for color, material, and local texture synthesis. Based on this observation, we first introduce Timestep-Conditioned Adapter Scaling (TCAS), the shared-layer form that replaces a globally fixed adapter scale with a stage-dependent scale function; TCAS is retained as the shared-layer baseline throughout the experiments. TCAS is not intended as a generic smooth guidance schedule. Its low-high-low form encodes the adapter-specific hypothesis that geometry residuals should be strongest only after coarse structure becomes stable and before late texture synthesis dominates. The layer-wise form defined next—assigning each depth group its own schedule—is the main method of this paper.

To avoid ambiguity caused by different raw timestep directions in different schedulers, we use sampling progress p to represent the denoising process, where $p = 0$ denotes the start from noise and $p = 1$ denotes the end of sampling close to the final image state. TCAS divides denoising into early, middle, and late stages:

$$s(p) = \begin{cases} s_e, & 0 \le p < 1/3 \\ s_m, & 1/3 \le p < 2/3 \\ s_l, & 2/3 \le p \le 1 \end{cases}. \tag{4}$$

The previously studied shared-layer C3 configuration is:

$$(s_e, s_m, s_l) = (1.25, 2.50, 1.25). \tag{5}$$

This design follows the hypothesis that early correction can disturb global formation, middle correction can strengthen geometric alignment, and late correction can compete with texture synthesis. The new layer-wise form makes this control explicit. For a UNet depth group $l \in \{\text{deep}, \text{middle}, \text{shallow}\}$, we use

$$\mathbf{h}'_{l,t} = \mathbf{h}_{l,t} + s_l(p_t)\, A_l(\mathbf{h}_{l,t}, \mathbf{G}), \tag{6}$$

where $s_l(p)$ can be fixed or stage-dependent. Concretely, the adapter residuals are injected through wrapped residual blocks of the three UNet up-blocks; the injection blocks map to depth groups as $\text{up}_0 \to$ deep, $\text{up}_1 \to$ middle, and $\text{up}_2 \to$ shallow, and each group carries an implementation cap (deep 3.0, middle 3.5, shallow 0.8) so that the effective scale is $\min(s_l(p_t), c_l)$. The representative layer-LHL configuration uses $(1.25, 2.50, 1.25)$ for deep and middle groups and $(0.50, 0.75, 0.50)$ for the shallow group; none of these values reaches a cap. Global schedules are clamped by the same caps, so the historical global fixed-low baseline is effectively shallow-capped at 0.8; the layer-fixed-low control isolates exactly this difference. The shared-input ablation also evaluates layer-fixed-low, layer-fixed-mean (the exact 17/16/17 step means of layer-LHL: 1.65 for deep/middle and 0.58 for shallow), and layer-LLH controls. These controls are essential because they distinguish the benefit of reducing shallow residuals from the benefit of placing a high scale in a particular denoising stage.

The method introduces no new trainable parameters and does not modify the base model, VAE, or adapter weights; it only updates residual multipliers during inference. Because the adapter correction $A_l(\mathbf{h}_{l,t}, \mathbf{G})$ is computed whenever geometry conditioning is active, with or without scaling, the overhead of layer-wise scheduling is limited to per-step scale assignment (at most one multiplier per wrapped residual block) and does not add a network forward or measurable memory cost.

How the stage schedule is selected: the CAI interpretation. We use Correction–Alignment–Interference (CAI) as an empirical diagnostic, not as a theorem or a guaranteed selector. The stage utility notation $U_k = Q(\text{high scale only in } k) - Q(L)$ is useful for describing the results, but additive stage effects are only an approximation because denoising stages interact. The shared-input ablation shows that layer-fixed-mean and layer-LLH can outperform layer-LHL, so CAI does not identify a unique temporal schedule. Texture diagnostics are treated as variation indicators; fidelity is assessed separately with masked perceptual and color-error metrics. In the independent MV-Adapter audit, the pre-specified CAI rule is set-valued/undefined.

Remark 1. *The CAI interpretation is adapter-dependent. Changing the adapter, residual magnitude, layer semantics, or backbone requires re-estimating the stage utilities and repeating the paired comparison. Confidence intervals describe uncertainty in the measured utilities; they do not establish equivalence or non-inferiority. The independent-backbone experiment therefore tests whether stage-position effects are observable under another implementation, not whether C3, LHL, or layer-LHL is a universal choice.*

3.4. GeoTex-Adapter and FAC Adaptive Extension

GeoTex-Adapter, the scaling controls, and FAC correspond to three complementary dimensions of adapter control. GeoTex-Adapter determines what geometric residual correction should be injected. The shared-layer TCAS and the layer-wise form determine when and where the residual should be strengthened or weakened. FAC further modulates the residual in a fine-grained manner along spatial positions and frequency components.

GeoTex-Adapter is used as a fixed geometric residual backbone. It receives a 5-channel geometric input consisting of a 3-channel normal map, a 1-channel depth map, and a 1-channel foreground mask. A multi-scale geometric encoder produces a feature pyramid, and residual corrections are injected into multiple resolution levels of the UNet. The stage-aware scaling acts on the temporal and per-depth-group scale of these residuals to control the influence of geometric conditions without changing the adapter architecture.

To further examine whether stage-wise adapter control can be extended to learned adaptive correction, we also conduct experiments with Full Adaptive Correction (FAC). FAC contains three lightweight modules: LTAG learns timestep-dependent gating, GSG generates spatial gates from geometric features, and FSC performs frequency-selective correction on adapter residuals. Unlike the fixed schedules, FAC finely modulates

adapter residuals in the temporal, spatial, and frequency domains. In the extension experiments, FAC uses the shared-layer TCAS schedule as its base and further adds GSG and FSC for spatial and frequency correction.

It should be emphasized that FAC requires learning a small number of additional parameters, whereas the training-free scaling family (shared-layer and layer-wise) is the main method of this paper. FAC is used only as a learned adaptive extension; Section 4.6 reports a controlled re-examination that delimits when learned modulation can be expected to help.

4. Experiments

4.1. Experimental Setup

4.1.1. Experimental protocol and comparisons

Experiments are conducted using an MVPainter-style geometry-controlled multi-view diffusion pipeline. Each object contains a single reference image and corresponding 3D geometry. During preprocessing, the 3D geometry is rendered into target-view normal maps, depth maps, foreground masks, and ground-truth RGB images. The corrected round-two protocol uses six unique target views, $(0, 15, 12, 16, 13, 14)$ in the forward order (and $(14, 15, 0, 16, 12, 13)$ after reversal); the legacy order with a duplicated top view is retained only as an audit condition. All comparisons with ground-truth renderings use the same views, 256×256 resolution, and foreground masks.

Except for the FAC extension experiment, the saved clean-v2 scaling records keep the object list, fetched reference/conditioning batch, target views, PyTorch initial latent, base UNet, VAE, adapter weights, and number of denoising steps fixed within each object; only the adapter scale or schedule is changed. Those historical records predate explicit Python/NumPy dataset-side seed control, so they are not presented as exact-repeat determinism evidence. All new reruns use Python, NumPy, and PyTorch seeds fixed before dataset construction and before each object. The number of sampling steps is 50. The historical 24-object probe nominated C3 before the larger evaluation records were generated; the active round-two claims use the strict-276 clean-v2 panel and the separately labeled 276-object stage-placement follow-up. The probe-to-holdout protocol isolates the effect of adapter scaling from changes in model architecture, training data, sampling budget, or evaluation-set tuning.

Accordingly, the comparisons in this paper are comparisons among inference-time adapter scaling strategies under the same base pipeline, rather than a system-level benchmark among different texture generation architectures. Uniform adapter scales serve as the primary deployment baselines because they correspond to the most direct way of choosing a single fixed adapter strength at inference time. The probe variants further test whether the trade-off can be resolved by simple alternatives such as globally high scales, layer-wise redistribution, or moving high adapter strength to early, late, or multiple denoising stages. Stage-aware scaling changes only how this strength is scheduled across denoising stages and depth groups.

4.1.2. Evaluation sets and metrics

The source protocol records four evaluation sets: (1) a 50-object scale-sweep set; (2) a 26-object texture-audit set; (3) a 24-object probe set for the original schedule nomination; and (4) a 300-object evaluation pool containing a 276-object held-out subset. These exploratory records are retained for provenance; the active round-two results are the strict-276 clean-v2 panel, the separately labeled stage-placement follow-up, and the explicitly bounded extension audits.

Evaluation metrics are divided into three categories: shape and structure metrics (FG-SSIM, Edge-SSIM, and Crop-SSIM), signal and perceptual metrics (PSNR, masked LPIPS, and, where available, CIEDE2000), and texture-variation diagnostics (Laplacian variance, RGB standard deviation, and gradient magnitude). The three variation diagnostics are not direct measures of texture fidelity; their symmetric log-ratio errors relative to ground truth are reported separately. All image metrics are first computed over six unique target views and then averaged over views and objects. For texture-related conclusions, we report means, paired object-level bootstrap intervals, and win rates; DISTS is unavailable in the current baking environment and is not claimed.

The FAC extension uses the same object list and evaluation protocol as the 300-object evaluation pool, and its lightweight modules are trained only on the 1,706-object training pool for the strong-residual adapter instance, which is disjoint from all evaluation objects. TCAS is used as the main baseline, while LTAG, LTAG+GSG, and Full FAC are evaluated as ablation variants under the same paired protocol. The base model and original adapter remain frozen, and only lightweight adaptive correction parameters are learned.

4.1.3. Implementation and dataset details

All objects are drawn from the Objaverse 3D asset collection: ground-truth multi-view images are rendered offline with Blender (Cycles engine, 17 views at 256×256, white background), and the corrected target-view protocol uses the six unique camera IDs $(0, 15, 12, 16, 13, 14)$; the legacy duplicate-top order is retained only as an audit condition. The main adapter's training pool contains 1,118 rendered objects, while the separately documented strong-residual/FAC audit uses a 1,706-object pool from the same source. Both training pools are disjoint from the evaluation pool; we verified that each training/evaluation identifier intersection is zero. Within the 300-object evaluation pool, the 24-object probe set corresponds to the first 24 identifiers (obj_{0000}–obj_{0023}) and the remaining 276 identifiers (obj_{0024}–obj_{0299}) form a holdout that is strictly disjoint from the probe. The exploratory 50-object and 26-object records are retained as provenance; the active evidence freeze uses the strict 276-object cohort and its paired follow-up.

The base pipeline is an MVPainter-style geometry-controlled multi-view diffusion model built on a joint six-view latent UNet, loaded from the local `MVPainter_Pipeline` snapshot under `checkpoints/hf_repo` with diffusers 0.20.0, and used frozen together with its VAE. The pipeline is reference-image-conditioned; no text prompts are used. The GeoTex-Adapter takes a five-channel geometric input (3-channel normal map,

1-channel depth map, 1-channel foreground mask), produces a multi-scale residual pyramid, and injects corrections into multiple UNet resolution levels; for the adapter studied in the main tables, the requested scale is applied multiplicatively at every injected layer. Inference uses the Euler discrete scheduler with 50 denoising steps and a fixed random seed (42) shared by all schedules within an experiment, so all variants share the same initial latents; the model generates six views per object.

FG-SSIM, Edge-SSIM (edge masks derived from depth/normal discontinuities), PSNR, and LPIPS (AlexNet backbone) are computed on foreground-masked regions after background normalization; the three foreground variation diagnostics are computed on foreground pixels and are not interpreted as texture fidelity without their GT-relative errors. The main tables use GeoTex-Adapter `geotex_refattn_v1/geotex_step_0002000.pt` trained on the 1,118-object pool. The separate strong-residual/FAC records are labeled as an independent extension protocol and are not pooled with the clean-v2 main-adapter tables. Checkpoint and object-list hashes are recorded in the round-two evidence freeze and reproducibility package.

Statistical reporting. Object-level paired comparisons report 95% percentile bootstrap confidence intervals with 10,000 resamples, a fixed seed, and object win rates; the direction is reversed for lower-is-better metrics. A confidence interval crossing zero is described as no detected difference, not as evidence of equivalence or non-inferiority. Where a mean is accompanied by ±, the quantity is the standard deviation across objects. The stage-placement follow-up is labeled as such and is not treated as a blind confirmation.

4.2. Visible complete-object evidence

The numerical comparisons are accompanied by complete-object panels so that the practical trade-off is visible rather than inferred from a single proxy metric. Each panel uses the same reference-conditioned object and target view for the ground truth, no-adapter, conservative fixed-low, aggressive fixed-high, and C3 outputs. The examples deliberately include a clear improvement, a near tie, and a failure-boundary object; they are representative qualitative evidence, not a substitute for the paired 276-object statistics.

The historical local texture panels in the supplementary material are kept as diagnostics from their original protocol. They illustrate the motivation for stage-conditioned control, but are not relabeled as clean-v2 aggregate evidence. In particular, visual texture preservation is interpreted together with the fixed-low and LLH counterexamples in the current tables.

4.3. Shared-input layer-wise ablation

The ablation isolates three questions separately: (i) does redistributing residual strength across depth groups help at all (global vs layer-wise); (ii) does the choice of temporal pattern matter beyond each group's average strength (constant vs scheduled); and (iii) which temporal placements work (complete low/high pattern enumeration). We answer (i) and (ii) with a paired development ablation on 24 probe objects with three seeds, and (iii) with a complete binary factorial over all eight low/high temporal patterns under the same protocol. Each object was collated once per seed; all methods reused the same target, geometric features, initial latent, and condition-VAE seed. The full hash manifest and per-object rows are provided in Supplementary S9. Table 1 reports means over the 72 object-seed pairs. Both tables are development results on the probe set and are not relabeled as independent holdouts.

Table 1. Shared-input layer-wise ablation on 24 objects and three seeds. The primary metric is FG-LPIPS; lower is better.

Method	Full-PSNR↑	Full-LPIPS↓	FG-PSNR↑	FG-SSIM↑	FG-LPIPS↓	Edge-SSIM↑
Fixed-low	15.085	0.2922	8.125	0.3454	0.2195	0.4896
C3/LHL	16.058	0.2748	9.556	0.3929	0.2072	0.5063
Layer-fixed-low	16.861	0.3501	10.753	0.4249	0.1964	0.5238
Layer-fixed-mean	16.892	0.2796	11.179	0.4352	**0.1889**	0.5287
Layer-LHL	17.036	0.3143	11.209	0.4352	0.1928	0.5239
Layer-LLH	**18.165**	**0.2562**	**12.364**	**0.4467**	**0.1785**	**0.5438**

Table 2. Complete binary layer-wise temporal-pattern ablation on 24 probe objects and three seeds (development ablation; shared inputs per object–seed). FG-LPIPS is the selection metric; lower is better. Patterns with an early-high component are dominated on fidelity metrics.

Pattern	Full-PSNR↑	Full-SSIM↑	Full-LPIPS↓	FG-PSNR↑	FG-SSIM↑	FG-LPIPS↓	Edge-SSIM↑
LLL	20.801	0.8759	0.2200	16.020	0.6112	0.1124	0.5942
LLH	**21.856**	0.8832	0.1805	**17.466**	**0.6219**	**0.1046**	**0.6131**
LHL	20.561	0.8784	0.2008	15.661	0.6038	0.1193	0.5887
LHH	21.413	**0.8837**	**0.1734**	17.171	0.6194	0.1093	0.6113
HLL	18.433	0.8636	0.2246	14.082	0.5951	0.1295	0.5849
HLH	19.321	0.8721	0.1860	15.382	0.6132	0.1206	0.6089
HHL	18.155	0.8675	0.2092	13.747	0.5894	0.1355	0.5796
HHH	18.828	0.8731	0.1828	15.064	0.6114	0.1248	0.6073

The comparison separates layer redistribution from temporal placement. First, every layer-wise control improves foreground fidelity over global fixed-low (layer-LHL vs fixed-low: paired +3.083 dB FG-PSNR and −0.0267 FG-LPIPS, 95% intervals excluding zero), so the layer-wise redistribution itself—not only the temporal pattern—carries the gain. Second, the temporal placement matters but is metric-dependent: layer-fixed-mean is better than layer-LHL on FG-LPIPS, and layer-LLH is better than layer-LHL on every metric in this development comparison. We therefore use layer-LHL as a representative transfer record rather than as a uniquely selected optimum. Figure 4 shows the complete six-view output for one paired object; the full per-object set and hash records are in Supplementary S9.

4.3.1. Complete binary temporal-pattern ablation

To rule out that the two schedules above were a favourable pair, we enumerate all eight binary temporal patterns over the same two layer-wise scale levels (low: deep/middle 1.25, shallow 0.50; high: deep/middle 2.50, shallow 0.75) on the same 24 probe objects and three seeds, with the same shared-input pairing (576 object–seed–pattern records; Supplementary S10). Schedule names read early/middle/late, e.g. LLH places the high level only in the late stage. Development-only selection uses FG-LPIPS; the holdout is never used to choose a pattern.

Three observations follow from Table 2. First, late-stage high scale helps: every pattern ending in H beats the same prefix ending in L on FG-LPIPS (e.g. LLH 0.1046 vs LLL 0.1124; LHH 0.1093 vs LHL 0.1193). This is the opposite of the shared-layer case, where a late high scale was harmful, and it is the layer-wise counterexample to importing the C3 intuition. Second, an



Fig. 2. Complete-object comparison for a representative object. The same reference and view are shown for all conditions. The shared-layer C3 schedule reduces the over-control visible in fixed-high while retaining geometric alignment; the panel is qualitative evidence and is not selected as a best-case metric example.



Fig. 3. Failure-boundary examples from the same complete-object protocol. The left object is the baking/exporter boundary case and the right object has very low raw texture coverage. They are retained to show where the tested schedules do not resolve source-fusion or coverage limitations.

early high scale hurts: all early-low patterns beat their early-high counterparts. Third, the originally motivated layer-LHL is not the best pattern: layer-LLH (paired FG-LPIPS difference +0.0147, 95% CI [+0.0117, +0.0187]), layer-LHH (+0.0101, [+0.0080, +0.0123]), and the constant layer-LLL (+0.0069, [+0.0042, +0.0108]) all beat it, while LHL beats every early-high pattern (e.g. −0.0101 vs HLL, CI [−0.0129, −0.0074]). Layer-LLH is the strongest pattern on five of the seven reported metrics and layer-LHH on the remaining two; layer-LHL is best on none.

The ranking is, however, a development-probe ranking. Two independent shared-input probe runs of overlapping schedules show that the LLH-over-LHL direction is stable in both, whereas the small differences among {LLL, LHL, layer-fixed-mean} change sign between runs because the reference-image preprocessing draw differs across runner invocations (Supplementary S10). We therefore claim only the robust directions—late-high helps, early-high hurts, layer-wise beats global fixed-low—and we confirm the development-selected pattern on the strict holdout below.

4.4. Strict holdout and stage-placement evidence

The current main-adapter evidence is reported in two explicitly separated panels. The first is the strict-276 slice of the clean-v2 four-condition run (no adapter, fixed-low, fixed-high, and C3), evaluated with the pre-save float metric path. The second is a targeted stage-placement follow-up on the same 276 object IDs, with fixed-mean, HLL, LHL, and LLH generated in one run. The repeated C3 outputs are not identical across these two runners (for example, the mean absolute Full-SSIM difference on the common objects is 0.00395), so absolute values are not pooled across panels. All paired comparisons within a panel use the same object IDs and seed.

Table 3. Strict-276 main-adapter clean-v2 panel. Metrics are pre-save float-evaluation means over the same six unique views; the Full-SSIM column is retained as an audit value and is not silently substituted for the saved-artifact branch.

Method	Full-PSNR↑	Full-SSIM↑	Full-LPIPS↓	FG-PSNR↑	FG-SSIM↑	FG-LPIPS↓	Edge-SSIM↑
No adapter	10.514	0.726	0.627	8.776	0.478	0.207	0.479
Fixed-low	15.086	0.850	0.194	7.030	0.355	0.202	0.500
Fixed-high	13.365	0.823	0.190	5.410	0.229	0.210	0.494
C3 (LHL)	14.875	0.855	0.190	6.763	0.348	0.201	0.500

The clean-v2 panel is a non-dominance result: C3 is above fixed-high on foreground PSNR/SSIM and is close to fixed-low on foreground LPIPS and Edge-SSIM, but fixed-low has higher Full-PSNR, FG-PSNR, and FG-SSIM. Thus the main run does not support a blanket claim that the shared-layer schedule improves practical image quality over the conservative fixed-low baseline.

The layer-wise LHL transfer record uses the same checkpoint, strict-276 object list, unique6 cameras, and 50-step protocol, but was produced by an independent candidate runner that, as later audits found, did not seed the Python RNG governing the reference-image preprocessing draw; a restart of that runner would not reproduce its exact outputs. It is therefore reported as an independent holdout record, not as a paired replacement for the clean-v2 four-condition table, and all holdout conclusions below rest on the seeded, same-runner confirmation set (Section 4.4.1). Its means are Full-PSNR 22.052,



Fig. 4. Visible layer-wise comparison for a representative development object. The panel restores the direct visual baseline chain requested in the previous revision: ground truth, global fixed-low, the shared-layer C3/LHL baseline, and layer-LHL. All columns use the same six target views and shared-input protocol. This is illustrative evidence; the paired 72-row statistics in Table 1 determine the reported comparison.

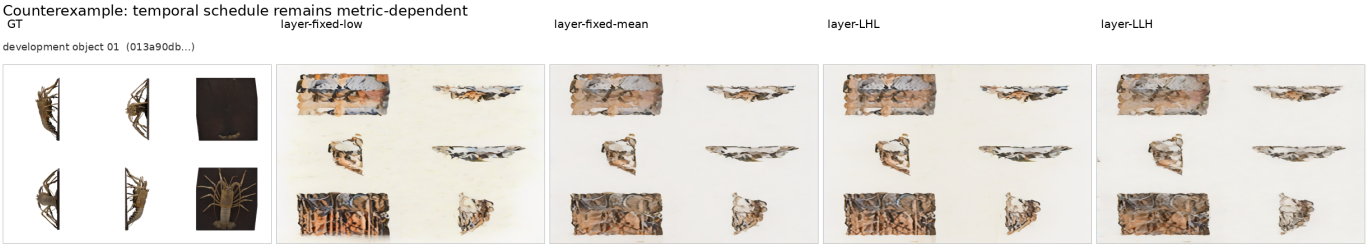


Fig. 5. Visible counterexample for the temporal schedule claim. The same development object is shown with ground truth, layer-fixed-low, layer-fixed-mean, layer-LHL, and layer-LLH. Layer-LHL is therefore not presented as a universally preferred schedule; this panel complements the paired metric counterexample in Table 1. The object was retained as a counterexample from the fixed seed-42 development render, not as a success-only selection.

Table 4. Stage-placement follow-up on the strict 276-object cohort. These are the six non-SSIM metrics from one paired run; the schedules are GLOBAL (one scale per stage applied to all depth groups), with 17/16/17 early/middle/late steps. Lower is better for LPIPS.

Schedule	Full-PSNR↑	Full-LPIPS↓	FG-PSNR↑	FG-SSIM↑	FG-LPIPS↓	Edge-SSIM↑
Fixed mean	14.311	0.188	6.257	0.316	0.205	0.499
HLL	13.397	0.202	5.583	0.259	0.212	0.464
LHL (C3)	14.868	0.190	6.760	0.348	0.201	0.500
LLH	15.136	0.181	6.937	0.357	0.200	0.536

Table 5. Saved-artifact Full-SSIM in the stage-placement follow-up. Prediction is PNG-reloaded float32; GT is the original RGBA composited over white in float32. C3 deltas use object-level paired bootstrap with 10,000 resamples and seed 20260928.

Schedule	Full-SSIM↑	C3 paired difference
Fixed mean	0.87395	+0.00749
HLL	0.85993	+0.02151
LHL (C3)	0.88144	reference
LLH	0.88313	−0.00170

Full-SSIM 0.8841, Full-LPIPS 0.1857, FG-PSNR 14.776, FG-SSIM 0.5510, FG-LPIPS 0.1338, and Edge-SSIM 0.5533. The shared-input development ablation above confirms the direction of the foreground gain but also shows that layer-LHL is not the unique best temporal schedule.

The follow-up provides evidence that stage placement matters, but it does not establish LHL as a universal choice. LLH is better than LHL on all six metrics in Table 4 under their intended directions. This is the central counterexample to interpreting CAI as a unique LHL selector. Notably, HLL and LLH have identical nominal budgets (mean 1.675, squared-scale sum 157.8125, 16 high-scale steps each) and differ only in whether the high block sits early or late; LLH is better on all six non-SSIM metrics and on the saved-artifact Full-SSIM branch (0.88313 vs 0.85993), so the placement effect already appears under this budget-matched global pair. The nominal budgets are also not strictly equal: fixed mean has scale mean 1.6667 and squared-scale sum 138.8889; HLL and LLH have means 1.675 and squared-scale sum 157.8125; LHL has mean 1.650 and squared-scale sum 153.125. The recorded actual residual L2/RMS summaries show further stage-dependent differences, so the result is a stage-placement sensitivity study rather than a perfectly equal-effective-budget intervention.

The corresponding 95% CIs for C3 minus fixed mean, HLL, and LLH are $[+0.00674, +0.00826]$, $[+0.01984, +0.02319]$, and $[-0.00205, -0.00134]$, respectively. The last interval is a detected loss for C3 on this saved-artifact Full-SSIM branch, not evidence that the schedules are equivalent. A 12-object saved-tensor decomposition fixes the GT branch before comparing representations: A fp16 to A fp32 changes mean SSIM by +0.03198; prediction PNG quantization changes it by −0.00036; and GT PNG serialization changes it by −0.00007. These diagnostic effects are not used to retrofit the historical 300-object table.

Comparison with generic schedules.. The original-protocol schedule audit also includes linear warm-up and cosine bump controls, reported separately in Supplementary S5. Those results are useful as a schedule-family comparison within the strong-residual audit, but they do not constitute a new clean-v2 experiment and are not pooled with Table 3. Thus, the evidence supports stage-aware scaling as a concrete alternative to

both fixed scales and common monotone/smooth schedules under the tested adapter, not as a universally superior schedule family.

4.4.1. Pre-registered confirmation on the strict holdout

Because the development probes select layer-LLH, we confirm the development-selected candidates on the strict 276-object holdout under a protocol written before any holdout number was observed (Section 4.3 candidates; Supplementary S11). The confirmation set re-runs layer-LLH, layer-fixed-mean, and a layer-LHL replica in one seeded runner (per-object seeds fixed for the reference preprocessing draw, initial latent, and condition-VAE path), so all comparisons below are same-runner paired. Table 6 reports the means; paired object-level 95% bootstrap intervals (10,000 resamples) are given in the text.

Layer-LLH is better than layer-fixed-mean on all seven metrics, with paired 95% intervals excluding zero: Full-PSNR +1.311 dB [+1.219, +1.400] (275/276 wins), FG-PSNR +0.875 dB [+0.769, +0.982] (240/276), FG-LPIPS −0.0089 [−0.0097, −0.0081] (258/276), Edge-SSIM +0.0113 [+0.0101, +0.0125], Full-LPIPS −0.0094, Full-SSIM +0.0011, and FG-SSIM +0.0070. Layer-LLH is also better than the layer-LHL replica on all seven metrics with intervals excluding zero: Full-PSNR +1.193 dB [+1.123, +1.262] (276/276 wins), FG-PSNR +1.001 dB [+0.907, +1.097] (261/276), FG-LPIPS −0.0110 [−0.0118, −0.0101] (264/276), Edge-SSIM +0.0203 [+0.0187, +0.0219], Full-SSIM +0.0028, Full-LPIPS −0.0290, and FG-SSIM +0.0174. Layer-fixed-mean versus the replica is mixed—fixed-mean is better on FG-LPIPS (+0.0021), FG-PSNR (+0.126), and Edge-SSIM (+0.0090), while the replica is better on Full-PSNR (+0.119)—matching the metric-dependent development picture. The replica's means are consistently lower than the archived unseeded layer-LHL record (e.g. FG-PSNR 12.97 vs 14.78; Pearson $r = 0.66$ over objects), confirming that the reference-preprocessing draw shifts absolute values between runners while the within-run orderings above are paired and seeded. These holdout results confirm the development-probe directions for the two promoted candidates; they do not re-open schedule selection, and the archived layer-LHL record (whose runner did not seed the reference-preprocessing draw and whose absolute values are not reproducible on restart) remains in the evidence package as an independent single-run record.

Table 6. Strict-276 same-runner confirmation of the pre-registered layer-wise candidates (seeded protocol; means over 276 objects). Absolute values are not comparable with Tables 3–4 (different runner invocations; different reference-preprocessing draws).

Schedule	Full-PSNR↑	Full-SSIM↑	Full-LPIPS↓	FG-PSNR↑	FG-SSIM↑	FG-LPIPS↓	Edge-SSIM↑
Layer-fixed-mean	19.516	0.8503	0.2082	13.100	0.4550	0.1698	0.5363
Layer-LHL (replica)	19.635	0.849	0.2278	12.974	0.445	0.1719	0.5273
Layer-LLH	20.828	0.8514	0.1988	13.975	0.4620	0.1610	0.5476

Table 7. Descriptive unseen-view baking metrics for the 12-object stratified case study. These values are not a population estimate; DISTS was unavailable.

Method	Masked PSNR↑	CIEDE2000↓	FG-LPIPS↓	Silhouette IoU
No adapter	11.824	18.200	0.156	0.99968
Fixed-low	6.001	48.623	0.199	0.99968
Fixed-high	3.891	60.420	0.216	0.99968
C3 (LHL)	5.638	51.183	0.201	0.99968

4.5. Baking and unseen-view case study

To test whether image-level differences survive into a real 3D output, we performed a CPU bake on the frozen, stratified 12-object Exact-GLB cohort. Each of the four main-adapter conditions produced a textured GLB, yielding 48 object–condition exports and 11 unseen-camera renders per export (528 view-level rows). The six source views were fixed and the unseen-view metrics were aggregated within object before any descriptive comparison.

This case study does not show a C3 advantage over fixed-low or no adapter for unseen-view fidelity: no adapter has the best descriptive PSNR, CIEDE2000, and LPIPS means in this cohort. The bake audit verifies semantic mesh/polygon/UV preservation and embedded textures, but it is not a byte-identity guarantee; Blender reindexed vertices for `obj_0048`. The zero seam values are reported with the audit's seam definition and denominator and are not treated as proof of universal seam-free output. Coverage and post-inpainting coverage are reported separately in the supplementary audit. The run has no generated 12-object GT bake and no DISTS result, and visual inspection found cross-view/source-fusion colour inconsistency. Thus the 3D result is a feasibility and limitation case study, not evidence of population-level superiority of any tested schedule superiority. Supplementary S4 gives object-level failure boundaries: `obj_0078` and `obj_0082` have raw coverage below 0.04, while `obj_0048` has 0.8441 inpainted fraction and is the exporter reindexing case.

Layer-wise bake extension with same-draw controls.. Because the reference-preprocessing draw shifts absolute values between runners, the layer-wise schedules were baked on the same 12 Exact-GLB objects together with fresh global fixed-low and C3 controls generated by the SAME seeded runner (identical per-object reference draws, latents, and seeds), so the four conditions are directly comparable. Within this same-draw set, layer-LLH beats layer-LHL on masked PSNR (16.679 vs 15.005), CIEDE2000 (14.186 vs 16.922), and FG-LPIPS (0.1215 vs 0.1420), with 12/12 paired object wins on all three metrics, and it also beats the same-draw global controls: layer-LLH vs global C3 is 16.679 vs 12.565 dB masked PSNR, 14.186 vs 21.643 CIEDE2000, and 0.1215 vs 0.1562 FG-LPIPS (12/12 object wins on all three), and the global C3-vs-fixed-low ordering is preserved (12/12 wins). The four means are strictly ordered (fixed-low 10.763 / C3 12.565 / layer-LHL 15.005 / layer-LLH 16.679 dB; CIEDE2000 26.287 / 21.643 / 16.922 / 14.186). This extension remains a stratified 12-object case study with the same limitations as above (no GT bake, no DISTS), so it is evidence about the layer-wise comparison, not a population-level 3D advantage claim.

4.6. FAC Adaptive Correction Extension: A Controlled Re-examination

The training-free schedules use fixed scale patterns, which have the advantages of requiring no training, being simple to

implement, and introducing low computational overhead. However, its residual scaling is still uniform across spatial positions and frequency components, lacking fine-grained adaptivity. To examine whether stage-wise adapter control can be extended to learned adaptive correction, we trained the FAC extension—LTAG for timestep gating, GSG for spatial gating, and FSC for frequency-selective correction on top of the TCAS temporal schedule. The base model, VAE, and GeoTex-Adapter remain frozen; only the three lightweight controllers are trainable, adding roughly 6×10^3 parameters. The controllers are trained with the same enhanced denoising objective as the base adapter—a latent noise-prediction MSE with 1.7× foreground and 1.3× edge spatial reweighting plus a latent-SSIM term—using AdamW (peak learning rate 10^{-4}, 100 warm-up steps) for 2,000 steps on the adapter's training pool, which is strictly disjoint from all evaluation objects, and all variants are evaluated under the separately documented strong-residual, per-layer-capped audit protocol with identical objects, seeds, and evaluation code.

In this controlled setting, no learned variant reached the training-free schedule. We tested seven configurations spanning joint, envelope-preserving, fidelity-weighted, and gently scheduled controllers. Their paired deficits relative to the training-free schedule range from −1.1 to −2.9 dB foreground PSNR (FG-PSNR; win rates at most 16%). A warm-start control in which the gate is left untrained at the exact C3 envelope shows no detected difference from the fixed schedule (paired FG-PSNR $\Delta = -0.18$ dB on the 12-object control, 95% CI $[-0.42, +0.05]$, crossing zero); this interval does not prove equivalence or validate transfer. No configuration or hyperparameter was selected on any evaluation object; the complete dose–response study is provided in the supplementary material.

For transparency, we note that an initial evaluation of the FAC extension, conducted on the main adapter under our earlier protocol before the strictly paired and strictly disjoint training controls were adopted, had suggested a positive gain. That initial comparison differed in both evaluation protocol and adapter instance, and it does not meet the controls we now consider necessary for a transfer claim; under the strictly paired, strictly disjoint protocol above, no trained configuration replicates the gain. We therefore report only the controlled re-examination and treat the initial comparison as superseded.

The outcome delimits the tested extension and does not identify a causal mechanism: every trained configuration degraded the measured fidelity under this protocol, while the untrained envelope control did not show a detected difference. Learning a modulation that improves upon the fixed layer-wise controls therefore requires a training signal that protects foreground fidelity; this is future work. The FAC results are reported as a negative, separate extension study, not as a claim that the fixed schedules are the only viable methods.

4.7. Second-backbone audit and contribution boundary

We audited the stage-position question on an independent MV-Adapter implementation using an Exact-GLB cohort of 76 objects and 11 recorded conditions. The full table and paired bootstrap output are provided in the reproducibility package.

Table 8. Selected within-backbone MV-Adapter means on the Exact 76-object cohort. Absolute scores are not compared with the MVPainter-style backbone.

Method	PSNR↑	FG-SSIM↑	Edge-SSIM↑	FG-LPIPS↓	ΔE_{00} ↓	GT-texture↓
No geometry	13.114	0.498	0.233	0.183	20.720	1.701
Fixed-low	13.357	0.528	0.240	0.188	20.150	2.224
Fixed mean	13.337	0.528	0.240	0.188	20.197	2.203
HLL	13.321	0.527	0.240	0.188	20.243	2.232
LHL	13.344	0.528	0.240	0.188	20.182	2.269
LLH	13.353	0.528	0.240	0.188	20.157	2.199

Dynamic schedules with high scale 1.50 and 1.00 are kept as separate scale families; their rows are not pooled. The table below shows the paired schedule comparison and the conservative controls.

One boundary must be stated before reading the table: this audit replicates the *global* stage-position experiment on the second backbone. The MV-Adapter deployment applies one scalar to all injected residuals (ControlNet-style condition scaling), and although the injected residuals are technically a per-block list that could be scaled element-wise, a layer-wise mapping would additionally require a depth-group allocation, a recalibrated low/high pair, and a full rerun; this has not been done. The audit is therefore a global stage-position replication, not a cross-backbone validation of the layer-wise method, and the MVDiffusion deployment remains an interface diagnostic only.

The audit supports an observable stage-position effect within this backbone, but not a uniform practical gain. LHL has slightly higher FG-SSIM and Edge-SSIM than HLL and LLH in the paired schedule comparison, while LLH has the highest PSNR among those four and lower GT-relative texture error than LHL. LHL has no clear advantage over fixed-low or fixed mean under the paired confidence intervals. The frozen CAI rule is `undefined_set_valued`, and the official MV-Adapter pretraining UID list was unavailable, so pretraining disjointness is not asserted. We therefore treat this result as a within- backbone diagnostic, not as a cross-backbone causal validation or a unique schedule-selection result.

5. Limitations

The contributions and the limits of this paper are those of a training-free inference control studied on one adapter family. First, the layer-wise evidence is bounded: the shared-input development ablation and the complete binary factorial are 24-object probe studies, and only the development-selected pattern (plus its layer-LHL representative) has strict-276 evidence. The development ranking is not a holdout ranking; small differences among constant-scale variants changed sign between two independent probe runs, so we claim only directions that held in both runs and on the holdout.

Second, the temporal-placement comparisons are not strict equal-effective-budget interventions. The 50-step partition is 17/16/17 and the nominal scale sums, squared sums, and actual residual norms differ across schedules; an equal-nominal-budget global check was started but interrupted before completion and lacks full checkpoint provenance, so it is kept as an internal record and budget-matched controls remain future work.

Third, the method's practical 3D validation is limited in scope. The original 12-object Exact-GLB bake exercises the four global conditions, and a same-draw extension adds the layer-wise schedules with fresh global controls; within that extension layer-LLH beats layer-LHL on all unseen-view metrics with 12/12 paired wins. The bake remains a stratified case study with no generated GT bake, no DISTS, and observed cross-view/source-fusion colour inconsistency; it must be read as an operational feasibility case study, not as a 3D quality advantage of any schedule.

Fourth, the second-backbone audit replicates only the global stage-position experiment; a layer-wise cross-backbone validation would require a per-block scale interface, a depth-group allocation, and a recalibrated comparison on the MV-Adapter deployment, which we have not performed. Pretraining-UID disjointness for the official MV-Adapter release is unknown.

Fifth, Laplacian variance, RGB standard deviation, and gradient magnitude are variation proxies, not fidelity measures; we report them with GT-relative errors, but they cannot replace human evaluation, particularly for reflective, transparent, or highly stylized objects. Finally, the schedules have not been validated across sampler families, resolutions, or reference-image quality, and the reference-image augmentation draw makes absolute values comparable only within a single runner invocation; all cross-runner tables are labeled as independent protocol surfaces.

6. Conclusion

This paper presented layer-wise stage-aware adapter scaling, a training-free inference control that assigns each UNet depth group its own stage-dependent residual scale. The controlled evidence supports four conclusions. (1) Moving from a global scale to depth-group scales improves foreground fidelity over global fixed-low under shared inputs (paired +3.083 dB FG-PSNR, −0.0267 FG-LPIPS on the development probe). (2) Temporal placement within the layer-wise family matters, but the pattern is not unique: the complete binary factorial shows late-stage high scale helps and early-stage high scale hurts, with layer-LLH strongest on five of seven development metrics. (3) The originally motivated layer-LHL is a representative, not an optimum: it is beaten by layer-LLH, layer-LHH, and the constant layer-LLL on the probe, and its strict-276 record is retained as a transfer record with a same-runner confirmation rather than as a selection. (4) These effects are adapter- and metric-dependent: on the second backbone only the global stage-position effect was replicated, with mixed practical signs.

The defensible contribution is therefore a concrete, reproducible, training-free layer-wise residual-scaling method, a complete temporal-pattern ablation with explicit counterexamples, and a mapped evidence boundary: no unique CAI selector, no universal schedule, no layer-wise cross-backbone validation, and no population-level 3D quality claim. The results motivate treating adapter strength as a layer- and stage-dependent control variable whose useful schedule must be re-estimated for each adapter and protocol.

Data availability

The evaluation scripts, per-object metric records, protocol manifests, hash indexes, and the comparison figures generated in this study are available from the corresponding author on reasonable request. The released artifact set comprises: the clean-v2 object-list and checkpoint-hash manifests, the shared-input ablation and binary-factorial raw CSV/JSON records with SHA-256 indexes, the strict-276 record files and paired-comparison outputs, the CPU bake metadata and unseen-view metric tables, and the MV-Adapter audit CSV/JSON files. Trained checkpoints and the Objaverse-derived renderings are redistributed only under the terms of their original licenses and are available from the corresponding author under the same conditions. No supplementary video is part of this revision.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Liu R, Wu R, Van Hoorick B, et al. Zero-1-to-3: Zero-shot one image to 3D object. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[2] Shi R, Chen H, Zhang Z, et al. Zero123++: A single image to consistent multi-view diffusion base model. arXiv:2310.15110, 2023.

[3] Liu Y, Lin C, Zeng Z, et al. SyncDreamer: Generating multiview-consistent images from a single-view image. *International Conference on Learning Representations*, 2024.

[4] Long X, Guo Y C, Lin C, et al. Wonder3D: Single image to 3D using cross-domain diffusion. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[5] Shi Y, Wang P, Ye J, et al. MVDream: Multi-view diffusion for 3D generation. *International Conference on Learning Representations*, 2024.

[6] Wang P, et al. ImageDream: Image-prompt multi-view diffusion for 3D generation. arXiv:2312.02201, 2023.

[7] Xu J, et al. InstantMesh: Efficient 3D mesh generation from a single image with sparse-view large reconstruction models. *Advances in Neural Information Processing Systems*, 2024.

[8] Li P, Liu Y, Long X, et al. Era3D: High-resolution multiview diffusion using efficient row-wise attention. *Advances in Neural Information Processing Systems*, 2024.

[9] Chen D Z, et al. Text2Tex: Text-driven texture synthesis via diffusion models. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[10] Richardson E, Metzer G, Alaluf Y, et al. TEXTure: Text-guided texturing of 3D shapes. *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

[11] Poole B, Jain A, Barron J T, Mildenhall B. DreamFusion: Text-to-3D using 2D diffusion. *International Conference on Learning Representations*, 2023.

[12] Chen R, Chen Y, Jiao N, Jia K. Fantasia3D: Disentangling geometry and appearance for high-quality text-to-3D content creation. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[13] Shao M, Xiong F, Sun Z, Xu M. MVPainter: Accurate and detailed 3D texture generation via multi-view diffusion with geometric control. arXiv:2505.12635, 2025.

[14] Kynkäänniemi T, Aittala M, Karras T, et al. Applying guidance in a limited interval improves sample and distribution quality in diffusion models. *Advances in Neural Information Processing Systems*, 2024.

[15] Perla S R K, Zhang H, Mahdavi-Amiri A. Advances in neural 3D mesh texturing: A survey. *Computer Graphics Forum*, 2026, e70392.

[16] Guzmán J E, Mould D, Paquette E. Adaptive multiresolution exemplar-based texture synthesis on animated fluids. *Computers & Graphics*, 2026, 134: 104490.

[17] Lv X, Wu Z, Xu J, et al. Interpretable procedural material graph generation via diffusion models from reference images. *The Visual Computer*, 2025, 41(13): 11195–11205.

[18] Zeng X, Chen X, Qi Z, et al. Paint3D: Paint anything 3D with lighting-immersive texture generation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[19] Cheng W, Mu J, Zeng X, et al. MVPaint: Synchronized multi-view diffusion for painting anything 3D. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[20] Zhao Z, Lai Z, Lin Q, et al. Hunyuan3D 2.0: Scaling diffusion models for high resolution textured 3D assets generation. arXiv:2501.12202, 2025.

[21] Huang Z, et al. MV-Adapter: Multi-view consistent image generation made easy. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025.

[22] Liu Y X, Xie M S, Liu H Y, Wong T T. Text-guided texturing by synchronized multi-view diffusion. *SIGGRAPH Asia 2024 Conference Papers*, 2024.

[23] Xu Q, Han X G, Zhang L. WonderTex: Consistent-and-seamless texture generation with text-guided multi-view image diffusion models. *IEEE Transactions on Visualization and Computer Graphics*, 2026, 32(7): 5815–5825.

[24] Mou C, Wang X, Xie L, et al. T2I-Adapter: Learning adapters to dig out more controllable ability for text-to-image diffusion models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[25] Zhang L, Rao A, Agrawala M. Adding conditional control to text-to-image diffusion models. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[26] Ye H, Zhang J, Liu S, Han X, Yang W. IP-Adapter: Text compatible image prompt adapter for text-to-image diffusion models. arXiv:2308.06721, 2023.

[27] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

[28] Liu S Y, Wang C Y, Yin H, et al. DoRA: Weight-decomposed low-rank adaptation. *International Conference on Machine Learning*, 2024.

[29] Zhang Q, Zuo S, Liang C, et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *International Conference on Learning Representations*, 2023.

[30] Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 2020.

[31] Song J, Meng C, Ermon S. Denoising diffusion implicit models. *International Conference on Learning Representations*, 2021.

[32] Ho J, Salimans T. Classifier-free diffusion guidance. *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021. arXiv:2207.12598.

[33] Castillo A, Kohler J, Pérez J C, et al. Adaptive guidance: Training-free acceleration of conditional diffusion models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

[34] Rombach R, Blattmann A, Lorenz D, Esser P, Ommer B. High-resolution image synthesis with latent diffusion models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[35] Esser P, Kulal S, Blattmann A, et al. Scaling rectified flow transformers for high-resolution image synthesis. *International Conference on Machine Learning*, 2024.

[36] Black Forest Labs. FLUX: Flow matching for image generation. https://github.com/black-forest-labs/flux, 2024.

[37] Loshchilov I, Hutter F. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.

[38] Cherti M, Beaumont R, Wightman R, et al. Reproducible scaling laws for contrastive language-image learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[39] Nichol A, Dhariwal P, Ramesh A, et al. GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models. *International Conference on Machine Learning*, 2022.

[40] Saharia C, Chan W, Saxena S, et al. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in Neural Information Processing Systems*, 2022.

[41] Peebles W, Xie S. Scalable diffusion models with transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[42] Wang Z, Bovik A C, Sheikh H R, Simoncelli E P. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 2004, 13(4): 600–612.

[43] Zhang R, Isola P, Efros A A, Shechtman E, Wang O. The unreasonable effectiveness of deep features as a perceptual metric. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.